ACM-ICPC Team Reference Document

Skill++ (Gogolev, Drachev, Shafikov)

Contents

1 General **1**
1.1 C++ Template 1
1.2 Compilation 2
1.3 Check 2
1.4 How to solve problems 2

2 Math **2**
2.1 Simpson 2
2.2 Prime 2
2.3 Modulo 2
2.4 FFT 3
2.5 NTT 3
2.6 Gcd 3
2.7 Hungarian 4
2.8 Gauss 4
2.9 Determinant 4

3 Strings **4**
3.1 Suffix Array 4
3.2 Suffix Automaton 5
3.3 Prefix function 5
3.4 Z function 5
3.5 Aho Korasik 6

4 Geometry **6**
4.1 Manhatan Spanning 6
4.2 Structures 6
4.3 Segment intesection 7
4.4 Semiplane intesection 7
4.5 Line and circle 8
4.6 Two circles 8
4.7 Minkowski 8

5 Data Structures **8**
5.1 Fenwick 8
5.2 Long 8
5.3 Persistent treap 9
5.4 Segment updates treap 10
5.5 Treap 10

6 DP **11**
6.1 Knuth 11
6.2 Convex hull trick 11

1 General

1.1 C++ Template

```
#pragma GCC optimize("O3,unroll-loops")
#define _CRT_SECURE_NO_WARNINGS
#define ln '\n'
#define All(x) (x).begin(), (x).end()
#define Allr(x) (x).rbegin(), (x).rend()
#define Allf(x) x.begin() + 1, x.end()
#define fi(a, b) for (auto i = (a); i <= (b); i++)
#define fj(a, b) for (auto j = (a); j <= (b); j++)
#define fo(a, b) for (auto o = (a); o <= (b); o++)
#define fq(a, b) for (auto q = (a); q <= (b); q++)
#define fe(a, b) for (auto e = (a); e <= (b); e++)
#define fw(a, b) for (auto w = (a); w <= (b); w++)
#define fdi(a, b) for (auto i = (a); i >= (b); i--)
#define fdj(a, b) for (auto j = (a); j >= (b); j--)
#define fdo(a, b) for (auto o = (a); o >= (b); o--)
#define fdq(a, b) for (auto q = (a); q >= (b); q--)
#define fde(a, b) for (auto e = (a); e >= (b); e--)
#define fdw(a, b) for (auto w = (a); w >= (b); w--)
#define fx(A) for (auto &x : (A))
#define fy(A) for (auto &y : (A))
#define fz(A) for (auto &z : (A))
#define pb push_back
#define mp make_pair
#define ins insert
#define sz(x) (int)x.size()
#include <string>
#include <vector>
#include <list>
#include <map>
#include <set>
#include <deque>
#include <queue>
#include <stack>
#include <bitset>
#include <algorithm>
#include <sstream>
#include <iostream>
#include <iomanip>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <cstring>
#include <cmath>
#include <cstdarg>
#include <cassert>
#include <ctime>
#include <tuple>
#include <unordered_set>
#include <unordered_map>
#include <random>
#include <chrono>
#include <complex>
using namespace std;
#ifdef _DEBUG
#define LOCAL
#endif
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch
    ().count());
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef vector<bool> vb;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<ld> vd;
typedef vector<char> vc;
typedef vector<string> vs;
typedef vector<vb> vvb;
typedef vector<vi> vvi;
typedef vector<vll> vvll;
typedef vector<vc> vvc;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
template<typename A, typename B>
ostream& operator<<(ostream& os, pair<A, B> p) {
    os << "(" << p.first << ", " << p.second << ")";
    return os;
}
template<typename T>
ostream& operator<<(ostream& os, vector<T> v) {
    fi(0, sz(v) - 1) {
        os << v[i] << " ";
    }
    return os;
```

```
}
template<typename T>
ostream& operator<<(ostream& os, set<T> t) {
    for (auto z : t) {
        os << z << "␣";
    }
    return os;
}
template<typename T1, typename T2>
ostream& operator<<(ostream& os, map<T1, T2> t) {
    cerr << endl;
    for (auto z : t) {
        os << "\t" << z.first << "␣->␣" << z.second << endl;
    }
    return os;
}
#ifdef LOCAL
#define dbg(x) {cerr << __LINE__ << "\t" << #x << ":␣"
      << (x) << endl;}
#else
#define dbg(x) {}
#endif
#ifdef LOCAL
#define ass(x) if (!(x)) { cerr << __LINE__ << "\tassertion␣
      failed:␣" << #x << endl, abort(); }
#else
#define ass(x) assert(x)
#endif
void solve() {

}
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cout << setprecision(20);
    #ifdef LOCAL
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
    #endif
    solve();
}
```

1.2 Compilation

```
#far association
g++ --std=c++17 !.! -DLOCAL -O2 -Wall -Wl,--stack
      =67108864 -o !.exe
```

1.3 Check

```
:start
gen.exe
sol.exe
copy output.txt answer.txt
NAME.exe
fc output.txt answer.txt
if ERRORLEVEL 1 goto end
goto start
:end
```

1.4 How to solve problems

1. Math formulation
2. Using parameters
3. Go to theory
4. Way od solutions:
   (a) fix parameter
   (b) cases
   (c) see on answer
   (d) induction
   (e) hypothesis
   (f) finding of
   (g) simplification
   (h) reduction to other problem
   (i) reduction to classic problem
   (j) reduction of classic problem to our
   (k) modeling
   (l) generalization
   (m) contrexamples
   (n) full random
   (o) invariant

2 Math

2.1 Simpson

```
double a = 0, b = 1;
const int n = 100000000;
double s = 0;
double h = (b - a) / n;
fi(0, n) {
        double x = a + h * i;
        s += fun(x) * ((i==0 || i==n) ? 1 : ((i&1)==0) ? 2 :4);
}
s *= h / 3;
```

2.2 Prime

```
ll fun1(ll a, ll n) {
        ll t = n - 1;
        int q = 0;
        while(t % 2 == 0) {
                q++;
                t /= 2;
        }
        ll v = pown(a, t, n);
        fi(1, q) {
                ll w = mult(v, v, n);
                if(v != 1 && v != n -1 && w == 1)return true;
                v = w;
        }
        if(v > 1)return true;
        return false;
}

bool is_prime(ll n) {
        if(n < 2)return false;
        if(n == 2)return true;
        if(n % 2 == 0)return false;
        fi(1, 10) {
                ll a = next(1, n - 1);
                if(fun1(a, n))return false;
        }
        return true;
}
```

2.3 Modulo

```
#define mod 1'000'000'007
int add(int a, int b){
        return (a + b) % mod;
}
int mult(int a, int b){
        return (ll)a * b % mod;
}
int pown(int a, ll n){
        if(n == 0)return 1;
        if(n % 2 == 0){
                int x = pown(a, n / 2);
                return mult(x, x);
        }
        return mult(a, pown(a, n - 1));
}
int obr(int a){
        return pown(a, mod - 2);
}
```

2.4 FFT

```
using cd = complex<double>;
const double PI = acos(-1);
void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}
vi multiply(vi const& a, vi const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < sz(a) + sz(b) ) {
        n <<= 1;
    }
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vi result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    while(result.back() == 0)result.pop_back();
    return result;
}
```

2.5 NTT

```
pii fun(int lg) {
        int n = (next(0, MAX / (1 << lg) / 2) * 2 + 1) * (1 <<
                lg) + 1;
        while(!is_prime(n)) {
                n = (next(0, MAX / (1 << lg) / 2) * 2 + 1) * (1
                        << lg) + 1;
        }
        vi d;
        fj(1, n - 1) {
                if(j * j > n - 1)break;
                if( (n - 1) % j == 0) {
                        d.pb(j);
                        if(j * j != n - 1)d.pb((n - 1) / j);
                }
        }
        sort(All(d));
        d.pop_back();

        fj(2, n - 1) {
                bool f = 1;
                fx(d) {
                        if(pown(j, x, n) == 1) {
                                f = 0;
                                break;
                        }
                }
                if(f)return mp(n, j);
        }
        return mp(n, -1);
}
int LG = 19;
int MOD, G;
void fft(vi & a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }
        for (int len = 2; len <= n; len <<= 1) {

        int wlen = G;
        if(invert)wlen = dv(1, wlen, MOD);
        for(int i = len;i < (1 << LG);i <<= 1) {
                wlen = mult(wlen, wlen, MOD);
        }

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j];
                int v = mult(a[i+j+len/2], w, MOD);
                a[i+j] = add(u, v, MOD);
                a[i+j+len/2] = sub(u, v, MOD);
                w = mult(w, wlen, MOD);
            }
        }
    }
    if (invert) {
        int rn = dv(1, n, MOD);
        for (int & x : a)
            x = mult(x, rn, MOD);
    }
}
vi multiply(vi const& a, vi const& b) {
    vi fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < sz(a) + sz(b) ) {
        n <<= 1;
    }

    fa.resize(n);
    fb.resize(n);
        fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = mult(fa[i], fb[i], MOD);
    fft(fa, true);
    while(fa.back() == 0)fa.pop_back();
    return fa;
}
auto T = fun(LG);
MOD = T.first;
G = T.second;
G = pown(G, (MOD - 1) / (1 << LG), MOD);
```

2.6 Gcd

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
bool find_any_solution(int a, int b, int c, int &x0, int &y0, int
        &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }
```

```
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

2.7 Hungarian

```
vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
for (int i=1; i<=n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<int> minv (m+1, INF);
    vector<bool> used (m+1, false);
    do {
        used[j0] = true;
        int i0 = p[j0], delta = INF, j1;
        for (int j=1; j<=m; ++j)
            if (!used[j]) {
                int cur = A[i0][j]-u[i0]-v[j];
                if (cur < minv[j])
                    minv[j] = cur, way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j], j1 = j;
            }
        for (int j=0; j<=m; ++j)
            if (used[j])
                u[p[j]] += delta, v[j] -= delta;
            else
                minv[j] -= delta;
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    } while (j0);
}
#get answer
vector<int> ans (n+1);
for (int j=1; j<=m; ++j)
    ans[p[j]] = j;
```

2.8 Gauss

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a
       big number

int gauss (vector < vector<double> > a, vector<double> &
      ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}

//binary case
int gauss (vector < bitset<N> > a, int n, int m, bitset<N> &
      ans) {
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        for (int i=row; i<n; ++i)
            if (a[i][col]) {
                swap (a[i], a[row]);
                break;
            }
        if (! a[row][col])
            continue;
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row && a[i][col])
                a[i] ^= a[row];
        ++row;
    }
        // The rest of implementation is the same as above
}
```

2.9 Determinant

```
const double EPS = 1E-9;
int n;
vector < vector<double> > a (n, vector<double> (n));

double det = 1;
for (int i=0; i<n; ++i) {
    int k = i;
    for (int j=i+1; j<n; ++j)
        if (abs (a[j][i]) > abs (a[k][i]))
            k = j;
    if (abs (a[k][i]) < EPS) {
        det = 0;
        break;
    }
    swap (a[i], a[k]);
    if (i != k)
        det = -det;
    det *= a[i][i];
    for (int j=i+1; j<n; ++j)
        a[i][j] /= a[i][i];
    for (int j=0; j<n; ++j)
        if (j != i && abs (a[j][i]) > EPS)
            for (int k=i+1; k<n; ++k)
                a[j][k] -= a[i][k] * a[j][i];
}
```

3 Strings

3.1 Suffix Array

```
struct Suff_arr{
  char LW = 3;
  int A = 256;
  string s0;
  string s1;
  vi arr;
  vi cost;
  vi rcost;
  vector<vector<pair<pair<int,int>,int > > > e1;
  vector<vector<pair<pair<int,int>,int > > > e2;
  vector<pair<pair<int,int>,int > > t;
  void Sort(vector<pair<pair<int,int>,int > >& t){
    fi(0,sz(t) - 1){
      e1[t[i].first.second].pb(t[i]);
    }
    fi(0,sz(e1) - 1){
```

```
    fx(e1[i]){
      e2[x.first.first].pb(x);
    }
    e1[i].clear();
  }
  t.clear();
  fi(0,sz(e2) - 1){
    fx(e2[i]){
      t.pb(x);
    }
    e2[i].clear();
  }

}
void init(string s){
  s0 = s;
  s1 = s;
  s1 += LW;
  fi(0,sz(s0) - 1){
    if( (sz(s1) & (sz(s1) - 1)) == 0)break;
    s1 += s0[i];
  }
  vvi e(A + 1);
  fi(0,sz(s1) -1){
    e[s1[i]].pb(i);
  }
  fi(0,A){
    fx(e[i]){
      arr.pb(x);
    }
  }
  cost = vi(sz(arr));
  rcost = vi(sz(arr));
  fi(1,sz(arr) - 1){
    cost[i] = cost[i-1];
    if(s1[arr[i]] != s1[arr[i-1]]){
      cost[i]++;
    }
  }
  int SIZ = 2;
  e1.resize(sz(arr));
  e2.resize(sz(arr));
  while(SIZ <= sz(arr)){
    fi(0,sz(arr) - 1){
      rcost[arr[i]] = cost[i];
    }
    t.clear();
    fi(0,sz(arr) - 1){
      t.pb( mp( mp(rcost[arr[i]], rcost[ (arr[i] + SIZ / 2)%sz(
          arr)]), arr[i]) );
    }
    Sort(t);
    cost[0] = 0;
    arr[0] = t[0].second;
    fi(1,sz(arr) - 1){
      arr[i] = t[i].second;
      cost[i] = cost[i- 1];
      if(t[i].first != t[i - 1].first)cost[i]++;
    }
    SIZ *= 2;
  }

  vi arr2;
  fx(arr){
    if(x < sz(s0)){
      arr2.pb(x);
    }
  }
  arr = arr2;
}
vi LCP;
void init_lcp(){
  vi rarr(sz(arr));
  fi(0,sz(arr) - 1){
    rarr[arr[i]] = i;
  }
  LCP = vi(sz(arr));
  int len = 0;
  fi(0,sz(arr) - 1){
    if(rarr[i] == sz(arr) - 1){
      len = 0;
      continue;
    }
    len = max(0,len - 1);
    int x = i;
    int y = arr[rarr[i] + 1];
    while(x + len < sz(s0) && y + len < sz(s0) && s0[x + len
        ] == s0[y + len]){
      len++;
    }
    LCP[rarr[i]] = len;

  }

 }
};
```

3.2 Suffix Automaton

```
struct state {
        int len, link;
        map<char, int> next;
};
const int MAXLEN = 1'000'000;
state st[MAXLEN*2];
int siz, last;
void sa_init() {
        siz = last = 0;
        st[0].len = 0;
        st[0].link = -1;
        siz++;
}
void sa_extend (char c) {
        int cur = siz++;
        st[cur].len = st[last].len + 1;
        int p;
        for (p = last; p != -1 && !st[p].next.count(c); p = st[p].
            link) {
                st[p].next[c] = cur;
    }
        if (p == -1) {
                st[cur].link = 0;
        } else {
                int q = st[p].next[c];
                if (st[p].len + 1 == st[q].len) {
                        st[cur].link = q;
                } else {
                        int clone = siz++;
                        st[clone].len = st[p].len + 1;
                        st[clone].next = st[q].next;
                        st[clone].link = st[q].link;
                        for (; p != -1 && st[p].next[c] == q; p =
                            st[p].link)
                                st[p].next[c] = clone;
                        st[q].link = st[cur].link = clone;
                }
        }
        last = cur;
}
```

3.3 Prefix function

```
vi prefix_function(string s) {
        int n = sz(s);
        vi pi(n);
        fi(1, n - 1) {
                int j = pi[i - 1];
                while (j > 0 && s[i] != s[j]) {
                        j = pi[j - 1];
        }
                if (s[i] == s[j]) j++;
                pi[i] = j;
        }
        return pi;
}
```

3.4 Z function

```
vi z_function (string s) {
        int n = sz(s);
        vi z (n);
        for (int i = 1, l = 0, r = 0; i < n; i++) {
                if (i <= r) z[i] = min(r - i + 1, z[i - l]);
                while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
                        z[i]++;
        }
                if (i + z[i] - 1 > r) {
            l = i;
```

```
            r = i + z[i] - 1;
        }
        }
        return z;
}
```

3.5 Aho Korasik

```
struct Node {
        int leaf = -1;
        char c = '\0';
        Node* par = nullptr;
        map<char, Node*> nxt;
        Node* suf = nullptr;
        map<char, Node*> sufs;
        Node(char t, Node* p) {
                c = t;
                par = p;
        }
};
Node* root;
void add(string& s, int j) {
        Node* cur = root;
        fi(0, sz(s)- 1) {
                if(cur->nxt.count(s[i])) {
                        cur = cur->nxt[s[i]];
                } else {
                        Node* t = new Node(s[i],cur);
                        cur->nxt[s[i]] = t;
                        cur = t;
                }
        }
        cur->leaf = j;
}
Node* get_link(Node* a) {
        if(a->suf) return a->suf;
        if(a == root) {
                a->suf = root;
                return a->suf;
        }
        if(a->par == root) {
                return root;
        }
        char t = a->c;
        Node* cur = get_link(a->par);
        while(cur != root && !cur->nxt.count(t)) {
                cur = get_link(cur);
        }
        if(cur->nxt.count(t)) {
                cur = cur->nxt[t];
        }
        return a->suf = cur;
}
Node* go(Node* a,char c) {
        Node* cur = a;
        while(cur != root && !cur->nxt.count(c)) {
                cur = get_link(cur);
        }
        if(cur->nxt.count(c)) {
                cur = cur->nxt[c];
        }
        return cur;
}

string s;
ll n;
vector<string> a;
void init() {
        root = new Node('\0',nullptr);
        fi(1, n) {
                add(a[i], i);
        }
}
```

4 Geometry

4.1 Manhatan Spanning

```
// Returns a list of edges in the format (weight, u, v).
// Passing this list to Kruskal algorithm will give the
      Manhattan MST.
vector<tuple<long long, int, int>> manhattan_mst_edges(
      vector<point> ps) {
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every rotation
        sort(ids.begin(), ids.end(), [&](int i, int j){
            return (ps[i].x + ps[i].y) < (ps[j].x + ps[j].y);
        });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it !=
                  active.end();
            active.erase(it++)) {
                int j = it->second;
                if (ps[i].x - ps[i].y > ps[j].x - ps[j].y) break;
                assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
                edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y - ps[
                      j].y), i, j});
            }
            active[ps[i].x] = i;
        }
        for (auto &p : ps) { // rotate
            if (rot & 1) p.x *= -1;
            else swap(p.x, p.y);
        }
    }
    return edges;
}
```

4.2 Structures

```
const ld eps = 1e-6;
struct Point {
    ld x;
    ld y;
    Point operator+(Point B) {
        return Point{x + B.x, y + B.y};
    }
    Point operator-(Point B) {
        return Point{x - B.x, y - B.y};
    }
    Point operator*(double t) {
        return Point{x * t, y * t};
    }
    bool operator==(Point B) {
        return abs(x - B.x) <= eps && abs(y - B.y)<=eps;
    }
    Point norm() {
        ld U = sqrt(x * x + y * y);
        return Point{x / U, y / U};
    }
    ld len() {
        return sqrt(x * x + y * y);
    }
};

struct Line{
  ll A;
  ll B;
  ll C;
};

struct Rec{
  Line L;
  Seg sg;
  Point V;
};

struct Seg {
    Point a;
    Point b;
    Point to_vec() {
        return Point{b.x - a.x, b.y - a.y};
    }
};
ld dot(Point p1, Point p2) {
    return p1.x * p2.x + p1.y * p2.y;
}
ld dot(Point p1, Point p2, Point p3) {
    return dot(p2 - p1, p3 - p1);
}
ld cross(Point p1, Point p2) {
    return p1.x * p2.y - p2.x * p1.y;
}
ld cross(Point p1, Point p2, Point p3) {
```

```
    return cross(p2 - p1, p3 - p1);
}
```

4.3 Segment intesection

```
optional<Point> intersect(Seg s1, Seg s2) {
    if (fabs(cross(s1.to_vec(), s2.to_vec())) < eps) {
        return {};
    }
    double t1 = cross((s2.a - s1.a), s2.to_vec()) / cross(s1.
        to_vec(), s2.to_vec());
    double t2 = cross((s1.a - s2.a), s1.to_vec()) / cross(s2.
        to_vec(), s1.to_vec());
    if (t1 < -eps || t1 > 1 + eps) {
        return {};
    }
    if (t2 < -eps || t2 > 1 + eps) {
        return {};
    }
    return s1.a + (s1.b - s1.a) * t1;
}
```

4.4 Semiplane intesection

```
ld area(Point a, Point b, Point c) {
    return (ld)1.0 * ((a.x - b.x) * (a.y - c.y) - (a.x - c.x) * (a.y -
        b.y));
}
bool inside(Point A, Point B, Point C, Point p) {
    if (area(A, B, p) == 0 || area(B, C, p) == 0 || area(A, C, p)
        == 0)
        return false;
    ld v1 = area(A, B, p);
    ld v2 = area(B, C, p);
    ld v3 = area(A, C, p);
    ld S = area(A, B, C);
    v1 = abs(v1);
    v2 = abs(v2);
    v3 = abs(v3);
    S = abs(S);
    return abs(S - v1 - v2 - v3) <= eps;
}
Point find_bis(Point A, Point B, Point C) {
    auto v = A - B;
    auto v2 = C - B;
    v.norm();
    v2.norm();
    return v + v2;
}
ld get_acos(double x) {
    return acos(max(-1., min(1., x)));
}

Line make_line(Point a, Point b) {
    ll A = round(b.y - a.y);
    ll B = round(a.x - b.x);
    ll C = round(a.y * b.x - a.x * b.y);
    if (A < 0)
    {
        A *= (-1);
        B *= (-1);
        C *= (-1);
    }
    ll g = gcd(A, gcd(abs(B), abs(C)));
    A /= g;
    B /= g;
    C /= g;
    return Line{A, B, C};
}
Point intersect(Line a, Line b) {
    ld d = a.A * b.B - b.A * a.B;
    ld d1 = -a.C * b.B + a.B * b.C;
    ld d2 = -a.A * b.C + a.C * b.A;
    ass(abs(d) > eps);
    Point res = {d1 / d, d2 / d};
    ass(on_line(res, a));
    ass(on_line(res, b));
    return {d1 / d, d2 / d};
}
Point get_ort(Point p, Point p2) {
    return {p2.y - p.y, p.x - p2.x};
}
Point get_ort(Line a) {
    return {a.A, a.B};
}
const ld pi = get_acos(-1.0);
bool good(deque<Rec> &dq) {
    vector<Point> P;
    vector<Rec> v;
    while (sz(dq)) {
        v.pb(dq.back());
        dq.pop_back();
    }
    fi(1, sz(v) - 1) {
        P.pb(intersect(v[i - 1].L, v[i].L));
    }
    P.pb(intersect(v[0].L, v[sz(v) - 1].L));
    ld res = 0;

    P.pb(P[0]);
    ld s = 0, s2 = 0;
    fi(1, sz(P) - 1) {
        s += P[i].y * P[i - 1].x;
        s2 += P[i].x * P[i - 1].y;
    }
    res = abs(s - s2);

    return res > 0 + eps;
}
bool paral(Line a, Line b) {
    return a.A * b.B - a.B * b.A == 0;
}
bool fun(ll T) {
    if (T == 0)
        return true;
    vector<Rec> vec(n + 1);
    set<tuple<ll, ll, ll>> t;
    fi(1, n) {
        Point p = a[i];
        Point p2 = a[go(i, T + 1, n)];
        vec[i] = Rec{make_line(p, p2), Seg{p, p2}, get_ort(p,
            p2)};
        if (area(p, p2, {p.x + 1'000'000 * vec[i].V.x, p.y + 1'000'
            000 * vec[i].V.y}) > 0 + eps) {
            vec[i].V = vec[i].V * -1;
        }
        // vec[i].V.norm();
        // t.ins({vec[i].L.A,vec[i].L.B,vec[i].L.C});
    }
    // ass(sz(t)==n);
    vector<Rec> top, bot;
    fi(1, n) {
        if (vec[i].V.y > 0 + eps || (abs(vec[i].V.y) <= eps &&
            vec[i].V.x > 0 + eps)) {
            bot.pb(vec[i]);
        } else {
            top.pb(vec[i]);
        }
    }
    sort(All(top), [](Rec &a, Rec &b)
         { return area(a.V, b.V, {0, 0}) <= eps; });
    sort(All(bot), [](Rec &a, Rec &b)
         { return area(a.V, b.V, {0, 0}) + eps >= 0; });
    vec = vector<Rec>(1);
    fz(bot) vec.pb(z);
    reverse(All(top));
    fz(top) vec.pb(z);
    deque<Rec> dq;
    dbg(vec);
    fi(1, n) {
        while (sz(dq) >= 2) {
            auto rec = dq.back();
            dq.pop_back();
            auto rec2 = dq.back();
            auto P = intersect(rec.L, rec2.L);
            dbg(P);
            if (area(vec[i].sg.a, vec[i].sg.b, P) <= eps) {
                dq.push_back(rec);
                break;
            } else {
                continue;
            }
        }

        while (sz(dq) >= 2) {
            auto rec = dq.front();
            dq.pop_front();
            auto rec2 = dq.front();
            Point P = intersect(rec.L, rec2.L);
            dbg(P);
            dbg(area(vec[i].sg.a, vec[i].sg.b, P));
```

```
            if (area(vec[i].sg.a, vec[i].sg.b, P) <= eps) {
                dq.push_front(rec);
                break;
            } else {
                continue;
            }
        }
        dq.pb(vec[i]);
    }

    while (sz(dq) >= 3) {
        auto rec = dq.front();
        dq.pop_front();
        auto rec2 = dq.front();
        auto P = intersect(rec.L, rec2.L);
        dbg(P);
        if (area(dq.back().sg.a, dq.back().sg.b, P) <= eps) {
            dq.push_front(rec);
            break;
        } else {
            continue;
        }
    }

    while (sz(dq) >= 3) {
        auto rec = dq.back();
        dq.pop_back();
        auto rec2 = dq.back();
        auto P = intersect(rec.L, rec2.L);
        dbg(P);
        if (area(dq.front().sg.a, dq.front().sg.b, P) <= eps) {
            dq.push_back(rec);
            break;
        } else {
            continue;
        }
    }

    dbg(T);
    dbg(sz(dq));

    return sz(dq) > 2;
}
```

4.5 Line and circle

We say first circle center at point $(0, 0)$. Line is (A, B, C). Find $x_0 = \frac{-AC}{A^2+B^2}$

$y_0 = \frac{-BC}{A^2+B^2}$

If dist from (x_0, y_0) to $(0, 0)$ is bigger than r then answer is zero. Otherwise if it equals to r then answer is (x_0, y_0). Otherwise we have 2 solutions: $d = \sqrt{r^2 - \frac{C^2}{A^2+B^2}}$ $V = \sqrt{\frac{d^2}{A^2+B^2}}$ $a_x = x_0 + BV$ $a_y = y_0 - AV$ $b_x = x_0 - BV$ $b_y = y_0 + AV$

4.6 Two circles

We have to find all tangent lines to two circles. First circle is at $(0, 0)$.

We say first circle center at point $(0, 0)$. $x^2+y^2 = r_1^2$ $(x - x_2)^2 + (y - y_2)^2 = r_2^2$

$x^2 + y^2 = r_1^2$ $x(-2x_2) + y(-2y_2) + (x_2^2 + y_2^2 + r_1^2 + r_2^2) = 0$ Now we have new task to find intersection of line and circle: $Ax + By + C = 0$, $A = -2x_2$, $B = -2y_2$, $C = x_2^2 + y_2^2 + r_1^2 - r_2^2$

4.7 Minkowski

```
struct pt{
    long long x, y;
    pt operator + (const pt & p) const {
        return pt{x + p.x, y + p.y};
    }
    pt operator - (const pt & p) const {
        return pt{x - p.x, y - p.y};
    }
    long long cross(const pt & p) const {
        return x * p.y - y * p.x;
    }
};

void reorder_polygon(vector<pt> & P){
    size_t pos = 0;
    for(size_t i = 1; i < P.size(); i++){
        if(P[i].y < P[pos].y || (P[i].y == P[pos].y && P[i].x < P[
            pos].x))
            pos = i;
    }
    rotate(P.begin(), P.begin() + pos, P.end());
}

vector<pt> minkowski(vector<pt> P, vector<pt> Q){
    // the first vertex must be the lowest
    reorder_polygon(P);
    reorder_polygon(Q);
    // we must ensure cyclic indexing
    P.push_back(P[0]);
    P.push_back(P[1]);
    Q.push_back(Q[0]);
    Q.push_back(Q[1]);
    // main part
    vector<pt> result;
    size_t i = 0, j = 0;
    while(i < P.size() - 2 || j < Q.size() - 2){
        result.push_back(P[i] + Q[j]);
        auto cross = (P[i + 1] - P[i]).cross(Q[j + 1] - Q[j]);
        if(cross >= 0 && i < P.size() - 2)
            ++i;
        if(cross <= 0 && j < Q.size() - 2)
            ++j;
    }
    return result;
}
```

4.8 Convex hull

```
struct pt {
    double x, y;
};

int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}
bool ccw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o > 0 || (include_collinear && o == 0);
}

void convex_hull(vector<pt>& a, bool include_collinear = false
    ) {
    if (a.size() == 1)
        return;

    sort(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.x, a.y) < make_pair(b.x, b.y);
    });
    pt p1 = a[0], p2 = a.back();
    vector<pt> up, down;
    up.push_back(p1);
    down.push_back(p1);
    for (int i = 1; i < (int)a.size(); i++) {
        if (i == a.size() - 1 || cw(p1, a[i], p2, include_collinear))
            {
            while (up.size() >= 2 && !cw(up[up.size()-2], up[up.
                size()-1], a[i], include_collinear))
                up.pop_back();
            up.push_back(a[i]);
        }
        if (i == a.size() - 1 || ccw(p1, a[i], p2, include_collinear)
            ) {
```

```
            while (down.size() >= 2 && !ccw(down[down.size()
                -2], down[down.size()-1], a[i], include_collinear)
                )
                down.pop_back();
            down.push_back(a[i]);
        }
    }

    if (include_collinear && up.size() == a.size()) {
        reverse(a.begin(), a.end());
        return;
    }
    a.clear();
    for (int i = 0; i < (int)up.size(); i++)
        a.push_back(up[i]);
    for (int i = down.size() - 2; i > 0; i--)
        a.push_back(down[i]);
}
```

5 Data Structures

5.1 Fenwick

```
// Linear
vll fen;
void add(int p, ll val) {
    for(int i = p; i <= n; i = (i | (i + 1))) {
        fen[i] += val;
    }
}
ll sum(int p) {
    ll res = 0;
    for(int i = p; i >= 0; i = (i & (i + 1)) - 1) {
        res += fen[i];
    }
    return res;
}

// Matrix
vvll fen;
void add(int x, int y, ll val) {
    for(int i = x; i <= n; i = (i | (i + 1))) {
        for(int j = y; j <= n; j = (j | (j + 1))) {
            fen[i][j] += val;
        }
    }
}
ll sum(int x, int y) {
    ll res = 0;
    for(int i = x; i >= 0; i = (i & (i + 1)) - 1) {
        for (int j = y; j >= 0; j = (j & (j + 1)) - 1) {
            res += fen[i][j];
        }
    }
    return res;
}
```

5.2 Long

```
struct Long {
    int base = 10000;
    int len = 4;
    vi num = {1, 0};
    int operator[](int x) const {
        if (x > num[0]) return 0;
        return num[x];
    }
    int& operator[](int x) {
        if (x > num[0]) {
            num.resize(x + 1, 0);
            num[0] = x;
        }
        return num[x];
    }
};
Long operator+(const Long& A, const Long& B) {
    Long res;

    int n = max(A[0], B[0]);
    fi(1, n) {
        res[i] = A[i] + B[i];
        if (i > 1) {
            res[i] += res[i - 1] / res.base;
            res[i - 1] %= res.base;
        }
    }
    while(res[res[0]] >= res.base) {
        int m = res[0];
        res[m + 1] = res[m] / res.base;
        res[m] %= res.base;
    }
    return res;
}
Long operator*(const Long& A, const int& B) {
    Long res;
    int n = A[0];
    fi(1, n) {
        res[i] = A[i] * B;
        if (i > 1) {
            res[i] += res[i - 1] / res.base;
            res[i - 1] %= res.base;
        }
    }
    while(res[res[0]] >= res.base) {
        int m = res[0];
        res[m + 1] = res[m] / res.base;
        res[m] %= res.base;
    }
    return res;
}
Long operator*(const Long& A, const Long& B) {
    Long res;
    fi(1, A[0]) {
        if (A[i] == 0) continue;
        fj(1, B[0]) {
            if (A[i] * B[j] == 0) continue;
            int x = i + j - 1;
            res[x] += A[i] * B[j];
            while(x > 1 && res[x - 1] >= res.base) {
                res[x] += res[x - 1] / res.base;
                res[x - 1] %= res.base;
            }
        }
        while(res[res[0]] >= res.base) {
            int m = res[0];
            res[m + 1] += res[m] / res.base;
            res[m] %= res.base;
        }
    }
    return res;
}
pair<Long, int> operator/(const Long& A, const int B) {
    Long res;
    ll x = 1;
    ll d = 0, m = 0;
    fdi(A[0], 1) {
        ll val = m * res.base + A[i];
        d = val / B;
        m = val % B;
        if (x == 1 && d == 0) continue;
        res[x] = d;
        x++;
    }
    reverse(Allf(res.num));
    return mp(res, m);
}
void print(const Long& A) {
    printf("%d", A[A[0]]);
    fdi(A[0] - 1, 1) {
        printf("%04d", A[i]);
    }
    printf(ln);
}
Long str_to_Long(const string& s) {
    int n = sz(s);
    Long res;
    int x = 1;
    for(int i = n - 1; i >= 0; i -= 4) {
        int val = 0;
        for(int j = max(0, i - 3); j <= i; j++) {
            val = val * 10 + (s[j] - '0');
        }
        res[x] = val;
        x++;
    }
    return res;
}
Long int_to_Long(int num) {
    Long res;
```

```
    int x = 1;
    while(num) {
        int val = num % res.base;
        num /= res.base;
        res[x] = val;
        x++;
    }
    return res;
}
```

5.3 Persistent treap

```
int next(int l,int r){

        return l + rnd()%(r - l + 1);
}
const int MAX = 1e7;

int siz = 1;
vi Left(MAX,-1),Right(MAX,-1),Siz(MAX,0),Val(MAX,0);

void upd(int v){
        if(v == -1)return;
        Siz[v] = 1;
        if(Left[v] != -1)Siz[v] += Siz[Left[v]];
        if(Right[v] != -1)Siz[v] += Siz[Right[v]];
}

bool INIT;
int cop(int v){
        if(INIT)return v;
        int x = siz;
        siz++;
        Val[x] = Val[v];
        Left[x] = Left[v];
        Right[x] = Right[v];
        Siz[x] = Siz[v];
        return x;
}

int merg(int L,int R){
        if(L == -1)return R;
        if(R == -1)return L;

        if(next( 1, (Siz[L] + Siz[R])*100 ) <= Siz[L] * 100){
                L = cop(L);
                Right[L] = merg(Right[L], R);
                upd(L);
                return L;
        }else{
                R = cop(R);
                Left[R] = merg(L,Left[R]);
                upd(R);
                return R;
        }
}
pair<int,int > split(int v,int k){
        if(Siz[v] <= k){
                return mp(v,-1);
        }
        if(k == 0){
                return mp(-1,v);
        }
        v = cop(v);
        int siz = 0;
        if(Left[v] != -1)siz = Siz[Left[v]];
        if(siz >= k){
                auto t = split(Left[v],k);
                Left[v] = t.second;
                upd(v);
                return mp(t.first,v);
        }else{
                auto t = split(Right[v], k - siz - 1);
                Right[v] = t.first;
                upd(v);
                return mp(v,t.second);
        }

}

int root = -1;
void seg_copy(int i,int j,int k){
        auto t = split(root,j - 1);
        auto g = split(root,i - 1);
        auto h = split(g.second,k);
        auto f = split(t.second,k);
        root = merg(t.first,merg(h.first,f.second));
}

int n,m;
vi cnt,from,to;
vi res;

void solve() {
        INIT = true;
        fi(1, n){
                int v = siz++;
                upd(v);
                Val[v] = i;

                root = merg(root,v);
        }
        INIT = false;
}
```

5.4 Segment updates treap

```
const ll E = 0;

struct Node {
    ll val;
    ll buf;
    ll min_val;
    ll cnt;
    ll y;
    Node *l;
    Node *r;
};

void push(Node* a) {
        if(!a) return;
        if(a->buf == E) return;
    a->val += a->buf;
    a->min_val += a->buf;

    if(a->l) {
                a->l->buf += a->buf;
        // a->l->min_val += a->buf;
        }
        if(a->r) {
                a->r->buf += a->buf;
        // a->r->min_val += a->buf;
        }
        a->buf = E;
}
void upd(Node *a) {
    a->cnt = 1;
    a->min_val = a->val;
    if (a->l) {
        a->min_val = min(a->min_val, a->l->min_val + a->l
              ->buf);
        a->cnt += a->l->cnt;
    }
    if (a->r) {
        a->min_val = min(a->min_val, a->r->min_val + a->r
              ->buf);
        a->cnt += a->r->cnt;
    }
}
Node* new_node(ll val) {
        Node* x = new Node;
        x->y = rnd();
        x->val = val;
        x->buf = E;
        x->cnt = 1;
        x->l = x->r = 0;
        upd(x);
        return x;
}
Node *merge(Node *a, Node *b) {
    push(a);
    push(b);
    if (!a)
        return b;
    if (!b)
        return a;
    if (a->y > b->y) {
        a->r = merge(a->r, b);
        upd(a);
        return a;
    }
    b->l = merge(a, b->l);
```

```
    upd(b);
    return b;
}
pair<Node *, Node *> split(Node *a, ll q) {
    push(a);
    if (!a)
        return {0, 0};

    ll left = 0;
    if (a->l)
        left += a->l->cnt;
    if (left >= q) {
        auto [l, r] = split(a->l, q);
        a->l = r;
        upd(a);
        return {l, a};
    }
    auto [l, r] = split(a->r, q - left - 1);
    a->r = l;
    upd(a);
    return {a, r};
}
```

5.5 Treap

```
struct Node {
    ll val;
    ll cnt;
    ll y;
    Node *l;
    Node *r;
};
void upd(Node *a) {
    a->cnt = 1;
    if (a->l)
        a->cnt += a->l->cnt;
    if (a->r)
        a->cnt += a->r->cnt;
}
Node *merge(Node *a, Node *b) {
    if (!a)
        return b;
    if (!b)
        return a;
    if (a->y > b->y) {
        a->r = merge(a->r, b);
        upd(a);
        return a;
    }
    b->l = merge(a, b->l);
    upd(b);
    return b;
}
pair<Node *, Node *> split(Node *a, ll q) {
    if (!a)
        return {0, 0};
    ll left = 0;
    if (a->l)
        left += a->l->cnt;
    if (left >= q) {
        auto [l, r] = split(a->l, q);
        a->l = r;
        upd(a);
        return {l, a};
    }
    auto [l, r] = split(a->r, q - left - 1);
    a->r = l;
    upd(a);
    return {a, r};
}
```

6 DP

6.1 Knuth

```
int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];

    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };

    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }

    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k
                ++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] + cost;
                }
            }
            dp[i][j] = mn;
        }
    }

    return dp[0][N-1];
}
```

6.2 Convex hull trick

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = INF;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```